Energy-Aware Routing of Sentiment Analysis Between a Microcontroller and a Transformer Hub

Does learning to defer beat confidence routing on a battery-powered device?

Ehsan Abdulkhader 24BCT0089 · Danyson Antony 24BAI0196 · Aditya Achar 24BCI0250

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore

Three parts, three presenters, about 25 minutes

PART 1

The problem, the idea and the data

Aditya Achar

Slides 1–8

PART 2

Models, cost and learned deferral

Danyson Antony

Slides 9–15

PART 3

Results, energy and conclusions

Ehsan Abdulkhader

Slides 16–22

Battery devices need to understand text, but transformers don't fit on them

Social and assistive robots

Computing is a major battery drain: motors are only 16.6–54.6% of a robot's power

Off-grid messengers

People type straight into an ESP32-S3, e.g. Meshtastic on a LILYGO T-Deck

Wearables

A smartwatch that leans on the phone in your pocket as its hub

512 KB

of on-chip SRAM on an ESP32-S3

vs

66 million

parameters in DistilBERT, the smallest common transformer

Robot power: Liu et al. 2025; ESP32-S3 datasheet v2.2; DistilBERT: Sanh et al. 2019

Three lines of work meet here, and one gap is untested

Cascades

- Viola–Jones detector, 2001
- BranchyNet early exits, 2017
- Confidence cascades for NLP: Varshney & Baral, 2022
- Edge-cloud cascades: Huang et al., 2025

Learning to defer

- Selective classification: Geifman & El-Yaniv, 2017
- Calibration: Guo et al., 2017
- Learning to defer: Madras et al., 2018
- Consistent losses: Mozannar & Sontag, 2020

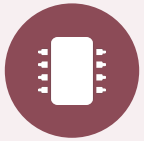
TinyML and energy

- EdgeBERT on custom accelerators, 2021
- Energy-aware multi-exit TinyML, 2026
- Small transformers lose accuracy: Turc et al., 2019
- NLP's energy cost: Strubell et al., 2019

The gap: learned deferral has not been tested from a microcontroller, where the deferral model itself must fit in 128 KB and the cost that matters is battery energy.

Full references in the paper (IEEE format, 36 sources)

A tiny model on the device, a transformer on the hub, and a rule for when to ask



Device: ESP32-S3, on battery

- Hash word 1–2-grams (no vocabulary stored)
- Chip model, 256 KB int8: p(x), 92.7% accurate
- Deferral model, 128 KB: g(x)
- Answer locally for about 0.8 mJ

review (Wi-Fi)

label



Hub: Raspberry Pi Zero 2 W

- 512 MB of RAM, about USD 15
- Int8 transformer: DistilBERT or RoBERTa-base
- 95.8–96.7% accurate
- Sends the label back

Send only if $g(x) > \lambda$ λ = what one Wi-Fi send costs, in accuracy; sweeping it traces accuracy vs. battery

Which reviews should the device send to the hub?

Confidence routing

Send the reviews the device is least sure about

- Score: 1 − max(p, 1 − p)
- Knows “I might be wrong”
- Free: no extra model

Learned deferral

Predict whether the hub would fix the device's answer

- Gain = hub right − device right (−1, 0, +1)
- g(x) = P(fix) − P(break), 128 KB model
- Trained on 300k held-out reviews

Hypothesis: learned deferral should skip reviews the hub would also get wrong, or that the device already gets right, so it reaches the same accuracy with fewer sends.

22.8 million scraped reviews, cleaned and split

1. Sources

- Official IMDb: 50,000 reviews
- Scraped IMDb (Kaggle): 5.5M
- Amazon Movies & TV: 17.3M

2. Label and clean

- IMDb: ≤ 4/10 negative, ≥ 7/10 positive
- Amazon: 1–2★ vs 4–5★
- Middle ratings dropped; duplicates removed
- Left: 4.1M IMDb, 12.0M Amazon

3. Split

- Test: official 25,000 (never changed)
- Train 20,000 / validation 5,000
- Pool training slice: 1,532,574
- Deferral slice: 300,000 (held out)
- val_pool: 50,000 unseen movies

Official IMDb splits from Maas et al. 2011; Kaggle IMDb review dataset (Biswas); Amazon Reviews 2023, Movies & TV

We removed every way test reviews could leak into training



4.1M

clean IMDb reviews in the extra pool

from 5.5M scraped (plus 12.0M Amazon)



23,818

copies of test or validation reviews removed

exact matches after normalising text



635,853

reviews by test-set authors removed

plus 163,409 reviews of test-set movies



0

deferral-slice reviews in hub training

an older slice overlapped 31%; we re-drew it

Official IMDb test set (25,000 reviews) is scored once; every threshold is chosen on validation data

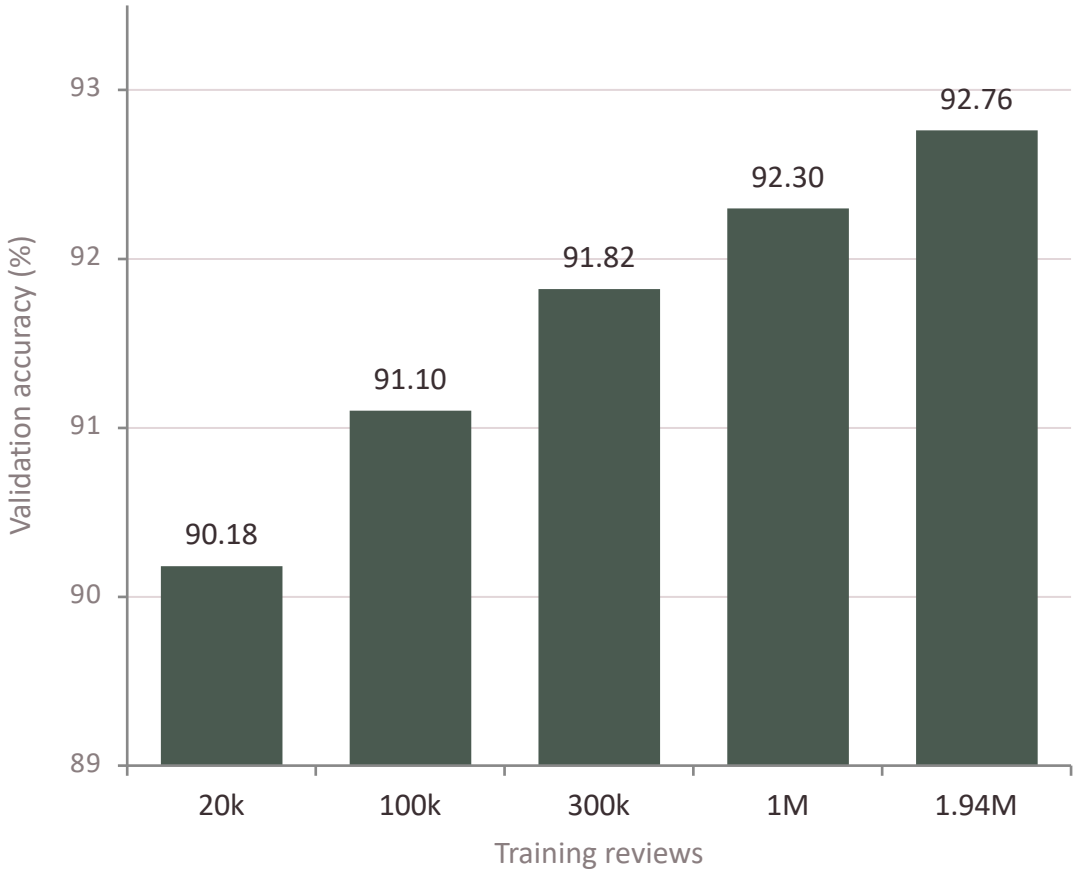
A 256 KB model that is just a table of 8-bit weights

How it reads a review

1. Split into words and word pairs
2. Hash each into one of 262,144 buckets
3. Add up the buckets' int8 weights, divide by √n
4. Sigmoid turns the sum into p(x)

$$p(x) = \sigma(0.3075 \times \Sigma\, q_b / \sqrt{n} + 0.0152)$$

Trained with Naive Bayes weighting and logistic loss, averaged SGD for 5 passes, then rounded to int8



Validation accuracy of the 256 KB int8 model (device v1) as the training set grows; Naive Bayes weighting: Wang & Manning 2012

Retraining the hub on one million extra reviews

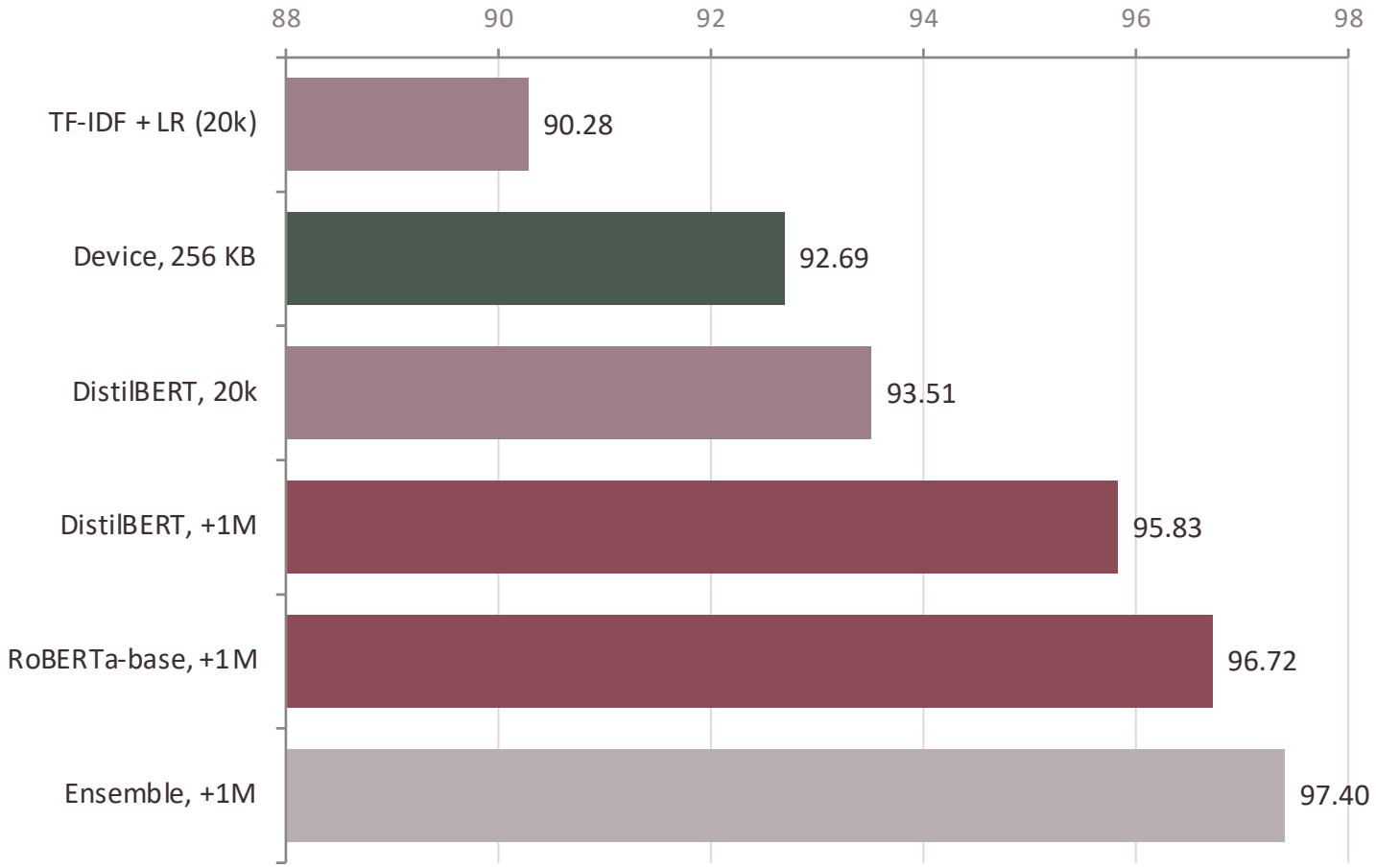
Hub	Training reviews	Epochs	Batch	Learning rate	Hardware	Speed	Cost
DistilBERT, 20k	20,000	2	16	3×10^{-5}	Laptop GPU (M3 Pro)	47 min total	free
DistilBERT, +1M	1,020,000	1	32	3×10^{-5}	NVIDIA H100	≈2,240 reviews/s	$0.91
RoBERTa-base, +1M	1,020,000	1	32	2×10^{-5}	NVIDIA H100	≈1,170 reviews/s	$1.64

Shared recipe for the +1M runs

- 1,020,000 ÷ 32 = 31,875 optimiser steps; AdamW, warm-up over the first 6%, then linear decay; bfloat16
- Reviews over 512 tokens keep their first 128 and last 382 tokens; similar lengths are batched together
- Every 2,000 steps, check accuracy on the 50k val_pool and keep the best checkpoint (never validation or test)

Settings from the saved run metrics; costs from Modal's billing report; H100 = $3.95 per hour

More leak-free training data lifted every model



+2.3 pts

DistilBERT with 1M extra reviews

97.40%

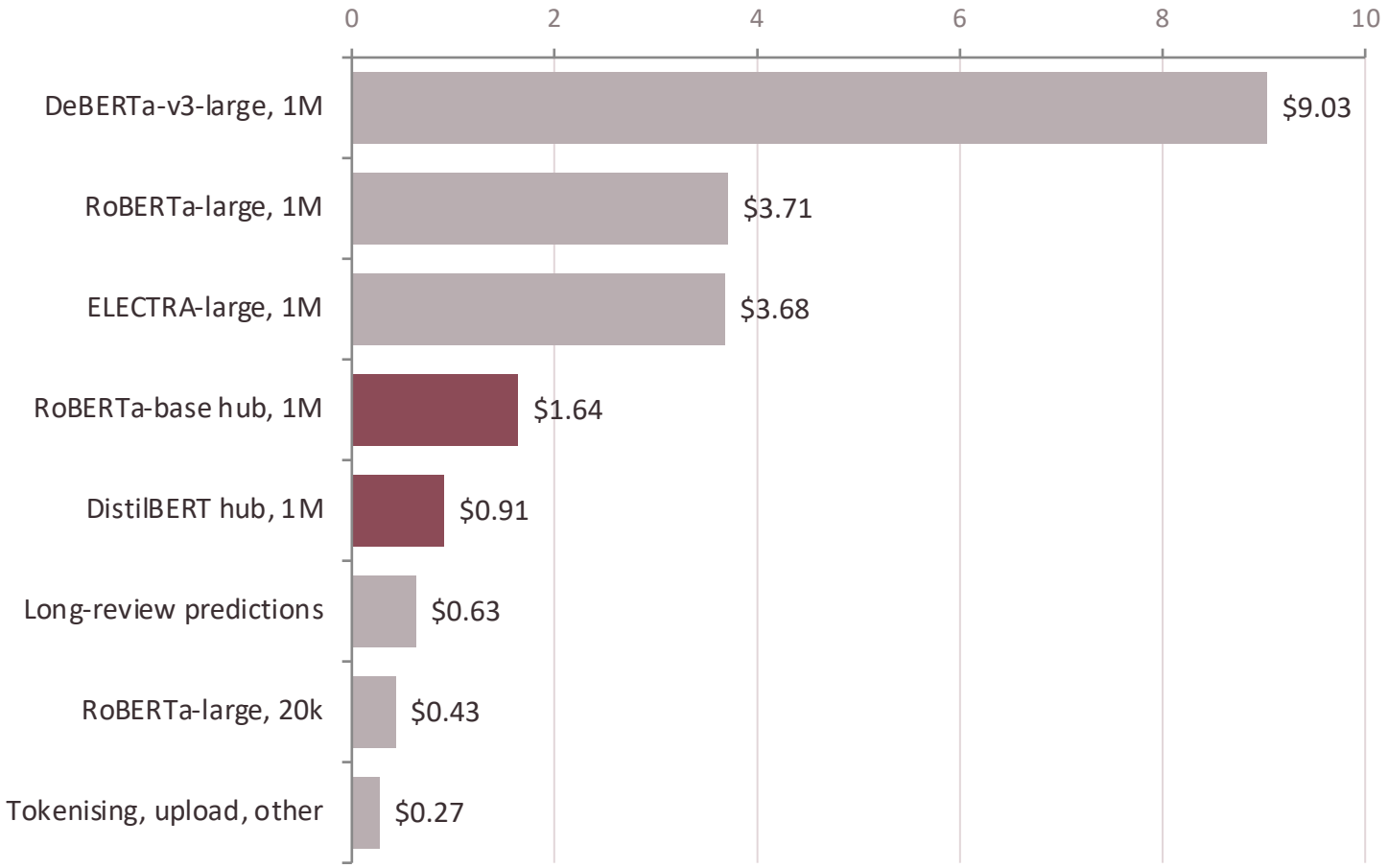
ensemble: above the best peer-reviewed 97.1%

24,180

of 25,000 test reviews right with RoBERTa-base

Official IMDb test set; the ensemble (ELECTRA-large + DeBERTa-v3-large + 1 MB n-gram) is a reference ceiling, too big for the hub

The whole project cost $20.27 of cloud GPU time



$2.56

retraining both hubs (paper)

$17.71

large record-attempt models

$0

laptop work: device model, routers, analysis

Itemised per run with `modal billing report`; paid from Modal credits, so $0 was billed. Every run had a hard time cap.

Learned deferral is trained on 300,000 reviews both models have answered

+1 **16,276**

hub fixes the device (5.43%)

0 **279,992**

no change (93.33%)

−1 **3,732**

hub breaks the device (1.24%)

What the router sees (fits the device)

- 65,536 hashed word buckets; two int8 tables = 128 KB
- Device confidence in 10 bins (0.5 to 1.0)
- Review length in 9 bins

How it is trained and used

- Gain = hub right − device right
- 3-class logistic regression on 250,000 reviews
- Score: g(x) = P(+1) − P(−1)

Counts for the RoBERTa-base hub on the re-drawn deferral slice; regularisation chosen at 10% sent (C = 0.1)

One operating point, from threshold to accuracy

1 **Pick the threshold on validation**

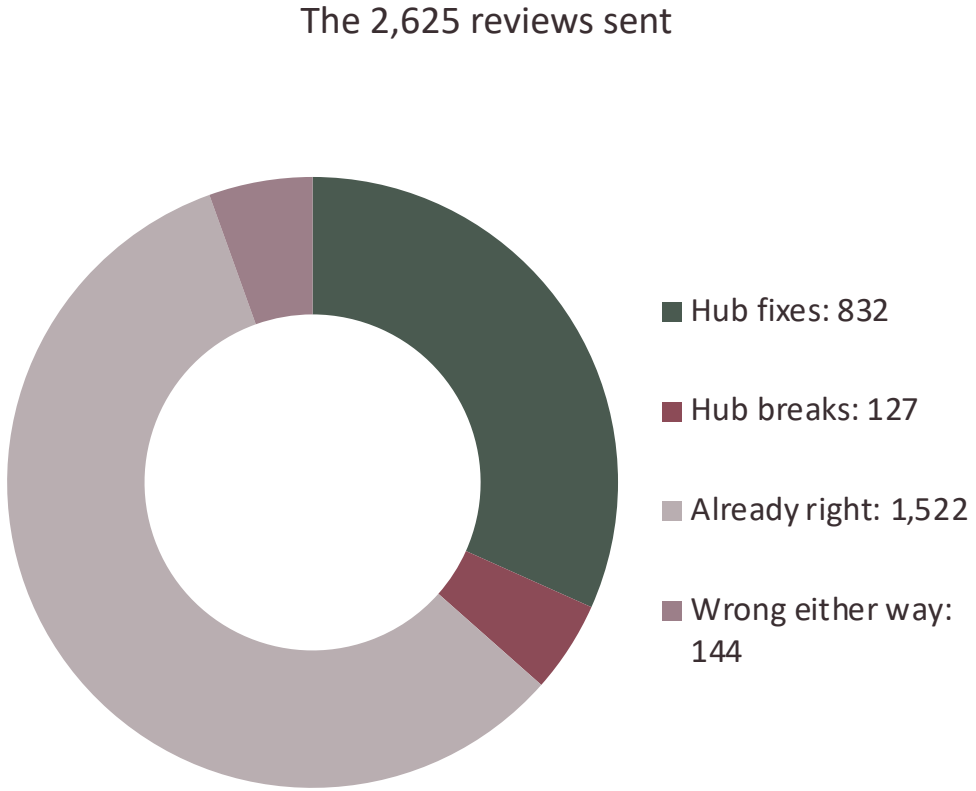
90th percentile of the 5,000 validation scores = 0.255, so send when the device's confidence is below 0.745

2 **Apply it to the test set once**

It sends 2,625 of 25,000 reviews (10.50%)

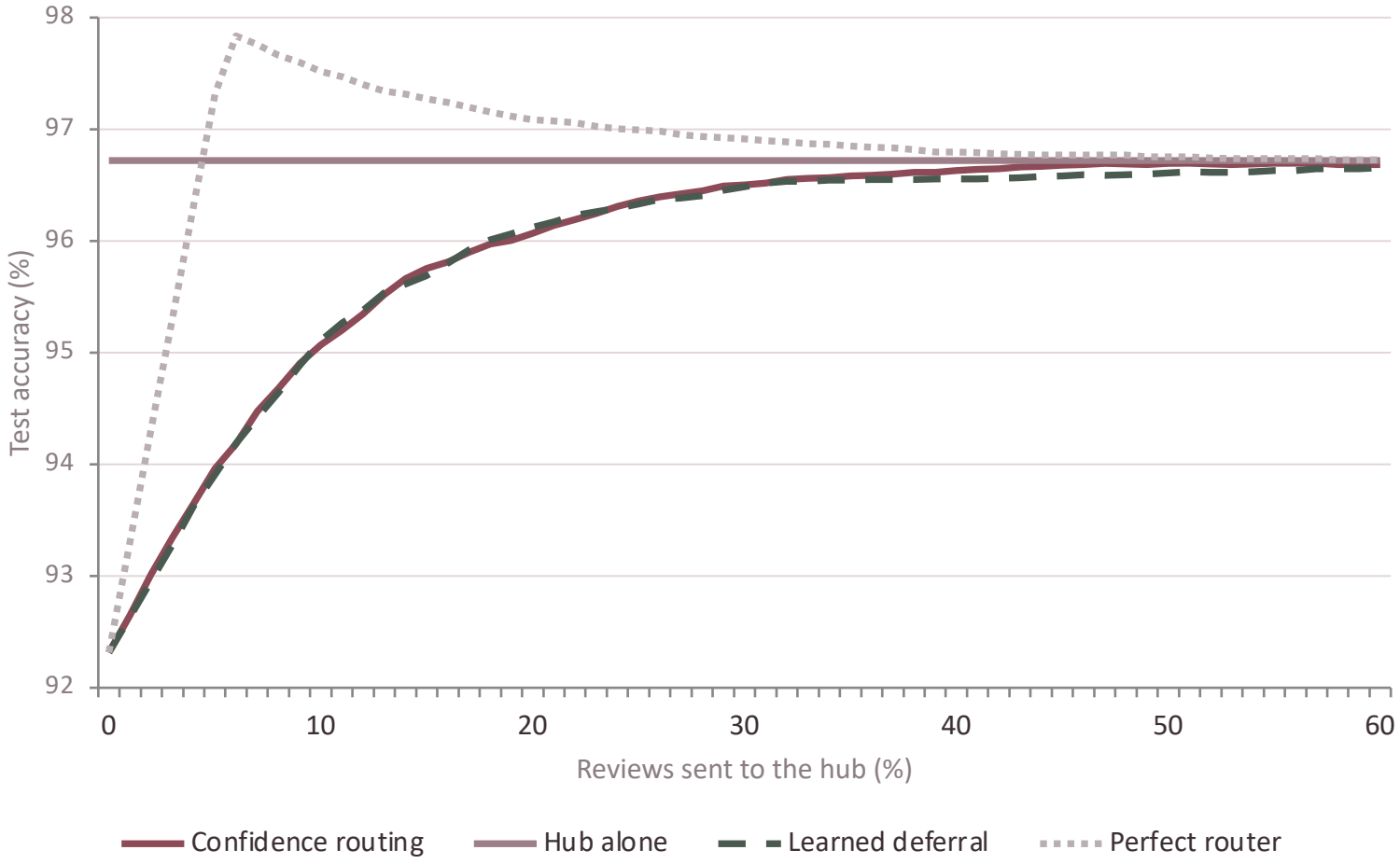
3 **Count right answers**

(23,079 + 832 − 127) ÷ 25,000 = 95.14%



RoBERTa-base hub, confidence routing, 10% target; device v2 alone gets 23,079 of 25,000 test reviews right

Learned deferral matches confidence routing, but doesn't beat it



≤ 0.14 pts

largest gap at the tested send rates, across 3 hubs

0 of 15

differences significant after Bonferroni correction

Hub: RoBERTa-base trained with 1M extra reviews; test-set curves (operating points on the next slide use validation thresholds)

The same picture holds for every hub we tried

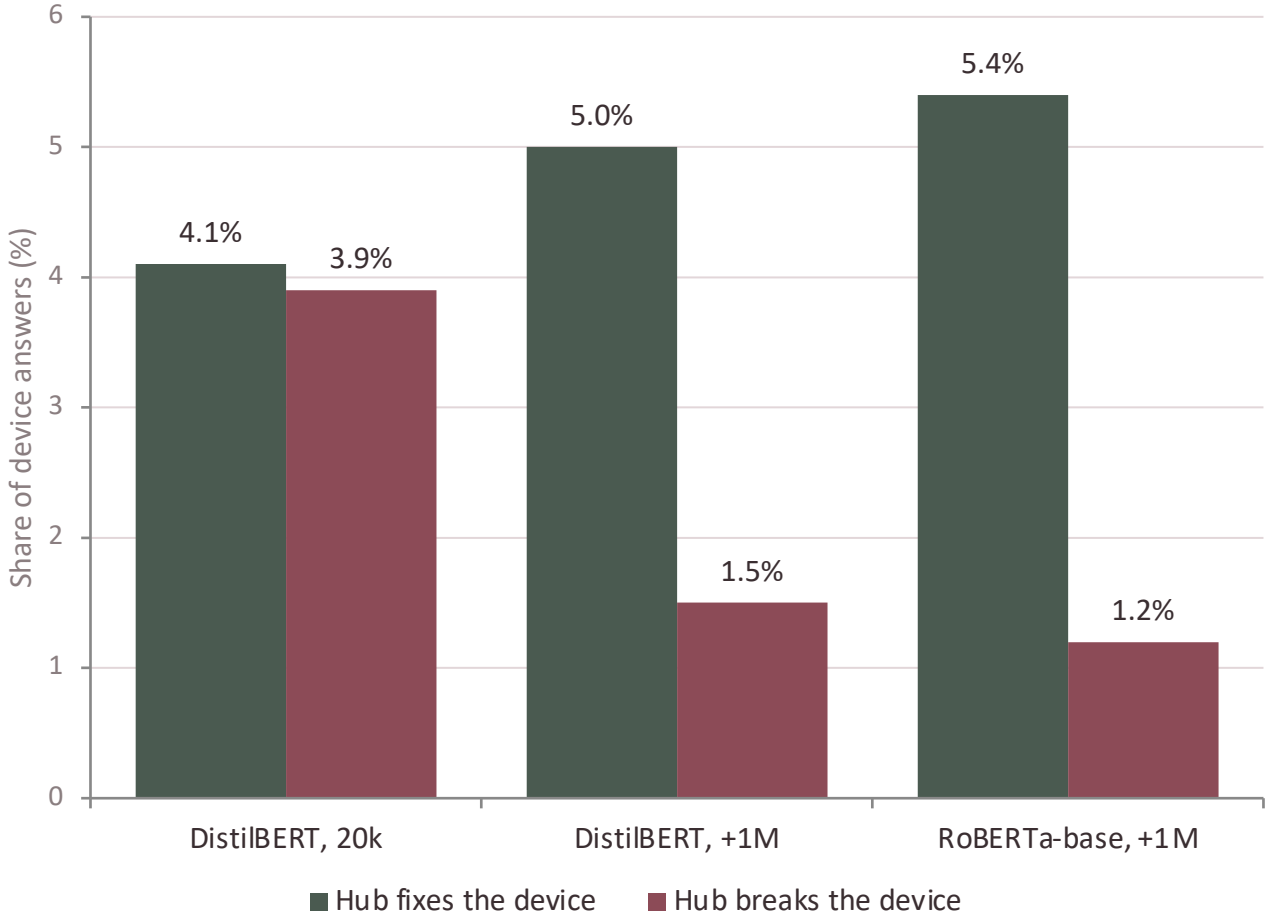
Hub (accuracy alone)	10% sent: confidence	10% sent: learned	p	20% sent: confidence	20% sent: learned	p
DistilBERT, 20k (93.51%)	94.05	93.92	0.04	93.91	93.98	0.28
DistilBERT, +1M (95.83%)	94.70	94.75	0.24	95.48	95.54	0.07
RoBERTa-base, +1M (96.72%)	95.14	95.23	0.03	96.12	96.16	0.28

The test, worked through (RoBERTa-base, 10% sent)

68 reviews are right only with learned deferral, 44 only with confidence routing. If the routers were equal these would split 50/50; seeing 44 or fewer of 112 has two-sided probability p = 0.029. That is above the corrected level of 0.0033, so it is not significant, and part of the gap comes from sending 10.7% instead of 10.5%.

Thresholds chosen on validation data; exact McNemar test on 25,000 paired test predictions; Bonferroni level 0.05 ÷ 15 = 0.0033

A stronger hub rarely breaks a correct answer, so there is little to avoid



58% already right

of what either router sends at 10%; learned deferral doesn't lower it

4.8% would break

so avoiding breaks, the easy win, is almost gone

97.3% possible

a perfect router at 5% sent: 128 KB of word buckets can't find which reviews the hub fixes

Shares of the 300,000-review deferral slice; breakdown at 10% sent is for the RoBERTa-base hub on the test set

Every send costs the device about 300 mJ; answering locally costs under 1 mJ

$$E(s) = V \times [\, I_{cpu} \times t_{local} + s \times (I_{tx} \times t_{tx} + I_{rx} \times t_{wait}) \,]$$

Answer on the device

3.3 V × 51.2 mA × 5 ms

0.84 mJ

Each send to the hub

3.3 V × (283 mA × 10 ms
+ 88 mA × 1 s)

299.74 mJ

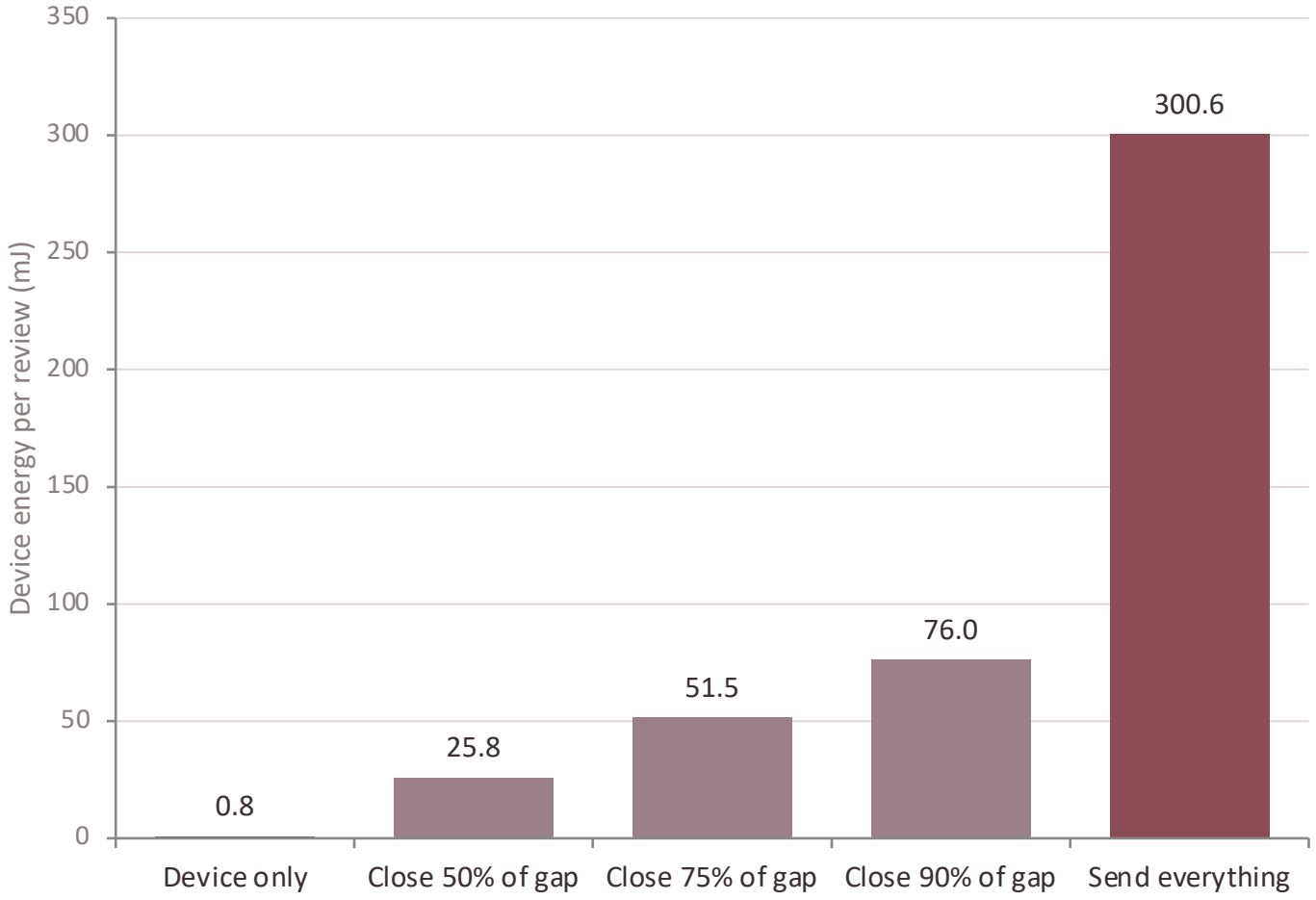
Send every review

0.84 + 1.0 × 299.74

300.58 mJ

Currents: ESP32-S3 datasheet v2.2 (240 MHz one core; 802.11n MCS7 at 18.5 dBm; RX). Timings are placeholders until measured.

Sending one review in six closes three quarters of the accuracy gap



Worked: close 75% of the gap

- Target on validation: 92.56 + 0.75 × 3.90 = 95.485%
- Needs 16% sent on validation
- On test: 16.9% sent, 95.89% accurate
- Energy: 0.84 + 0.169 × 299.74 = 51.5 mJ
- 300.58 ÷ 51.51 = 5.8× less than sending all

Targets chosen on validation data, then applied once to test; RoBERTa-base hub; 1 s hub wait (placeholder)

Closing three quarters of the gap gives almost 6× more reviews per charge than sending everything

Device only

31.5M

reviews per charge

0.8 mJ each · 92.32% accurate

Close 50% of gap

1.03M

reviews per charge

25.8 mJ each · 94.76% accurate

Close 75% of gap

517k

reviews per charge

51.5 mJ each · 95.89% accurate

Send everything

89k

reviews per charge

300.6 mJ each · 96.72% accurate

Reviews per charge = 26,640 J ÷ energy per review e.g. 26,640 ÷ 0.05151 J = 517,154

Illustrative 2,000 mAh, 3.7 V cell = 26,640 J; energy per review only (idle and sleep power not included)

What's left: measure it on real hardware

Limitations

- Energy is modelled: datasheet currents, placeholder timings
- Pi Zero 2 W latency and int8 model sizes not yet measured
- One dataset: long English movie reviews
- 0.1 points is only 25 of 25,000 test reviews

ESP32-S3 firmware

tokenise, hash, int8 lookup, route

Pi Zero 2 W hub

int8 ONNX model behind a small HTTP server

INA219 measurements

real latency and energy per review

Better router features

e.g. disagreement between two device models

Three takeaways

1

Confidence routing is enough

Learned deferral stayed within 0.14 points of it at every tested send rate, across three hubs.

2

Stronger hubs change the game

They rarely break device answers, which removes deferral's easy win.

3

Small sends, big savings

One review in six closes three quarters of the gap at 6–7× less energy.

Thank you · Questions?